

面向 LLM 服务的多阶段流调度

Yijun Sun¹ Xudong Liao¹ Songrun Xie¹ Hao Chen² Han Tian³
Wenxue Li¹ Yiming Zhang² Kai Chen¹

¹ 香港科技大学 iSING Lab

² 上海交通大学 ³ 中国科学技术大学

ABSTRACT

满足严格的首 Token 时间 (Time-To-First-Token, TTFT) 要求, 对 LLM 应用至关重要。为了提升效率, 现代 LLM 服务系统采用带有多样化并行方式的解耦架构, 引入了包含可复用 KV 块检索、集合通信以及 Prefill-to-Decode (P2D) 传输在内的复杂多阶段工作流。不同请求中具有依赖关系的各阶段流会在共享瓶颈链路上发生跨阶段、跨请求重叠, 使 TTFT 极易受到网络争用影响, 因此亟需具备阶段感知能力的调度机制。不幸的是, 现有大多数工作都以与阶段无关的方式调度流, 导致缺乏协调的争用, 进而成为 SLO 违约的主要原因之一。

本文提出 MFS, 一种旨在最大化 TTFT SLO 达成率的整体式多阶段流调度机制。其核心是在无需精确知道请求剩余松弛度的前提下, 近似实现 Least-Laxity-First (LLF) 调度策略。为此, MFS 通过反向多级队列 (Reverse Multi-Level Queue, RMLQ) 结构实现了一种延后并提升 (Defer-and-Promote) 原则。随着有效松弛度不断缩小, MFS 动态提升任务优先级, 从而优先保障松弛度更小的流, 同时避免 SLO 较宽松的请求过早占用网络带宽。我们将 MFS 实现为可插拔模块, 并将其集成到 vLLM 中, 在一个由 8 台服务器、32 块 GPU 组成的测试平台以及大规模仿真环境上进行了评估。结果表明, MFS 能够稳定超越当前最先进的基线方法, 将 TTFT SLO 达成率提升 1.2×–2.4×。

1 引言

大型语言模型 (LLM) 的快速发展催生了丰富的应用场景, 包括交互式聊天机器人 [20, 65, 101]、代码助手 [47, 105] 以及智能体系统 [27, 81, 95]。现代服务系统必须在维持海量请求吞吐的同时, 满足异构服务级目标 (SLO) [18, 29, 67, 76, 99], 以支撑严格的性能预期。在这些指标中, 首 Token 时间 (TTFT) 尤为关键, 因为它既决定了交互式用户所感知到的响应性 [25, 87], 也关系到智能体流水线中的严格超时约束 [37, 45, 57]。TTFT 目标被违反时, 往往会触发重复重试 [52, 53], 造成不必要的资源消耗 [70] 与显著营收损失 [73]。

现代 LLM 服务系统通常采用带 KV-cache 复用的 Prefill-Decode 解耦架构 [26, 31, 32, 66, 69, 79, 93, 106], 并利用多种并行方式 [9, 38, 46, 98] 来提升效率。尽管这些设计前景可观, 它们在 prefill 阶段仍然承受着显



Figure 1: 多阶段流调度与以往“与阶段无关”调度方式差异示意图。

著的通信开销。具体而言, 生成第一个 token 涉及多阶段通信: (1) 远程检索可复用的 KV 块; (2) 为中间激活同步而进行的集合通信; (3) Prefill-to-Decode (P2D) 传输。不同阶段的流经常会在共享网络上重叠并相互竞争, 从而产生请求内争用——同一请求内部不同通信阶段彼此干扰, 以及请求间争用——来自不同 batch 或不同 prefill 单元的通信竞争相同链路。这类争用会显著拉长 prefill TTFT 延迟, 并提升系统范围内 SLO 违约的风险。

遗憾的是, 现有工作在争用条件下无法满足端到端截止期限 (见图 1), 主要原因在于它们以与阶段无关的方式处理流 (即忽视阶段依赖)。传统流调度方案 (如 Fair Sharing [7, 107]、SJF [8, 10, 28, 41]、EDF [28, 83, 88] 等) 只针对单个流做决策, 缺乏整体视角。即使是 coflow 方案 [4, 21, 22, 86, 97, 100] 也存在不足: 它们虽然能把数据并行流作为一个组来管理, 却忽视了各阶段之间的内在依赖。因此, 这些方案可能过度优先早期阶段通信, 或让对时延敏感的下阶段被饿死, 最终损害 TTFT SLO 达成率。

基于此, 我们提出一个问题: 能否在 LLM 服务系统中以整体方式调度多阶段通信, 从而最大化总体 TTFT SLO 达成率?

在探索设计空间时, 我们注意到已有一些工作分别针对孤立阶段进行优化, 目标要么是集合通信 [15, 16, 72, 77, 91], 要么是 KV-cache 传输 [19, 51, 69, 79]。然而, 这些专用方案仍然不够。原因在于, 它们常常追求彼此冲突的优化目标, 若简单叠加, 反而可能放大网络争用。

本文给出的答案是肯定的。我们提出 MFS, 一种整体式多阶段通信调度器, 用于联合编排存在依赖关系的各通信阶段, 以最大化 TTFT SLO 达成率。我们的关键洞见是: 随着 prefill 执行推进, 全局 TTFT 截止时间可以逐步转化为显式的流级截止时间。基于这一观察, MFS 在不确定环境下近似 Least-Laxity-First (LLF) 策

略，并通过反向多级队列（RMLQ）实现延后并提升原则。通过依据有效松弛度的缩小动态提升任务优先级，MFS 能优先保障真正关键的阶段，同时避免 SLO 宽松的请求过早占用网络带宽。

尽管这一洞见本身并不复杂，但要把它落实为高效、可落地的真实系统，还需要解决若干非平凡挑战。首先，如何在不造成过度优先的前提下调度带显式截止时间的末阶段通信？其次，如何在松弛度不确定的情况下调度由隐式 TTFT 约束驱动的早期阶段流？第三，如何保证 MFS 与新兴集合通信库和 KV-cache 库兼容？

针对第一个挑战，我们采用一种以最小链路利用率（Minimal Link Utilization, MLU）为触发条件的惰性提升策略。P2D 流首先进入低优先级队列，只有当其 MLU 超过预设阈值时才被提升。这样可确保 P2D 流仅获得“恰好按时完成”所需的最小带宽，从而为早期阶段通信保留带宽空间。为了避免优先级抖动，提升操作仅在层粒度上发生。该方法既实现了有效的粗粒度优先控制，也便于在具备硬件优先级队列与轻量级包标记能力的商用交换机上落地，同时消除了分组乱序风险。

针对第二个问题，我们采用双层调度策略。对于单个 prefill 单元内部的请求内争用，我们优先调度能够解除后续计算阻塞的集合通信；只有当 KV-cache 传输可能阻塞下一层执行时，才提升其优先级。对于 prefill 集群中的请求间争用，我们根据 TTFT 截止时间对单元排序，并通过可行性检查剪除不可行的通信任务，以避免阻塞。该设计在松弛度不明确时避免了过早提升，同时确保早期阶段通信始终与下游 TTFT 需求保持一致。

最后，为了保证兼容性，MFS 通过轻量级任务适配器与现有软件栈（如 NCCL [60]、Mooncake [36]）集成。这些适配器会在主机端拦截任务，并在软件队列中执行精确优先级控制，同时将优先级映射为 DSCP 值，以在交换机上实现稳健的流量隔离。通过混合式优先级执行机制，MFS 能利用标准网络原语实现无缝集成与有效争用缓解。

我们构建了一个 MFS 原型，并在一个包含 8 台服务器、32 块 NVIDIA 3090 GPU [59]、16 张 Mellanox NIC [61]，且全部接入同一台 ToR 交换机的测试平台上进行了评估。我们将 MFS 实现为 NCCL 与 Mooncake 的可插拔模块，并将其接入 vLLM 推理引擎。基于该原型，我们在当前先进 LLM [3] 上验证了 MFS 的收益。

为了进一步评估 MFS 在大规模场景下的性能，我们还基于四类具有代表性的真实生产级 MoE 模型 [1, 2, 23, 71] 进行了大规模仿真。实验结果表明，与基线方案相比，MFS 可将 TTFT SLO 达成率提升 1.2×–2.4×。我们还观察到，MFS 将非重叠集合通信完成时间降低了 50%，并显著改善了请求提前量。

2 背景与动机

2.1 LLM 服务与 TTFT

大型语言模型已经成为现代 AI 服务的基础。这类模型天然具有自回归特性：系统首先处理完整提示词以生成

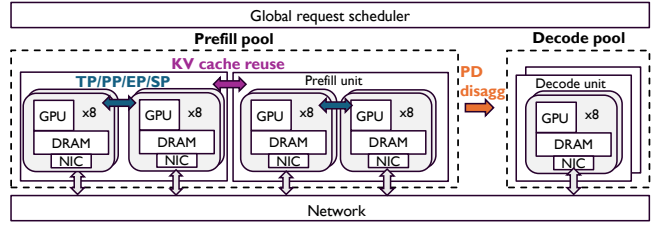


Figure 2: 大规模 LLM 服务示意图。

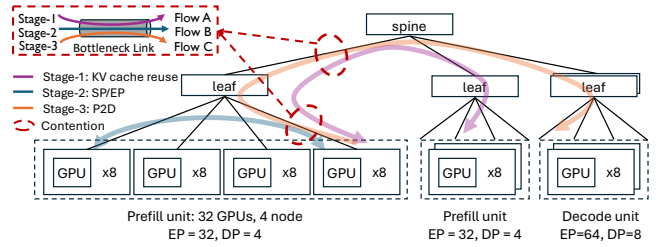


Figure 3: LLM 服务系统中的通信争用示例：KV cache 传输（阶段 1 和 3）与并行通信（阶段 2）竞争同一链路带宽。

第一个 token（prefill 阶段），然后利用累积得到的 KV cache 迭代地产生后续 token（decode 阶段）。其性能通常用 prefill 阶段的首 Token 时间（TTFT）和解码阶段的 token 间时间（TBT）来刻画。

TTFT 是许多 LLM 应用中的关键服务级目标（SLO）。在聊天机器人、语音助手等交互式场景中，用户期望系统能够立即给出初始响应；一些研究表明，当延迟超过数百毫秒时，用户就会明显失去耐心，而达到数秒级则可能直接放弃 [35, 49, 84]。TTFT 对自动化智能体框架同样重要，因为此类系统依赖严格超时 [37, 45] 来实现容错。首个 token 延迟会触发代价高昂的补救动作，例如服务重试 [52, 53] 或服务商切换 [57]。微软近期的一份报告 [73] 显示，首个 token 延迟是服务失败（以超时报错形式体现）的主要来源之一（约占 40%），并会带来收入损失、更高的支持成本和声誉风险。

如图 2 所示，现代生产系统 [46, 70] 通常扩展到由数千个节点组成的集群，这些节点通过高带宽链路互连，每个节点内部包含多个 xPU（如 GPU、TPU、NPU）。节点被组织成独立的服务单元，每个单元承载一个模型副本。模型部署会结合张量并行 [56]、流水线并行 [55, 102]、序列并行 [48, 89, 92] 和专家并行 [9, 38, 46, 98] 等方式。为进一步提升效率，近期工作 [32, 66, 79, 106] 提出 prefill-decode 解耦，将 prefill 任务部署在计算优化设备上，而将 decode 任务部署在内存更丰富的设备上，以适配其异构资源需求。此外，KV cache 复用 [26, 31, 69, 93] 也被引入，通过在节点间迁移缓存上下文来摊薄 prefill 成本。

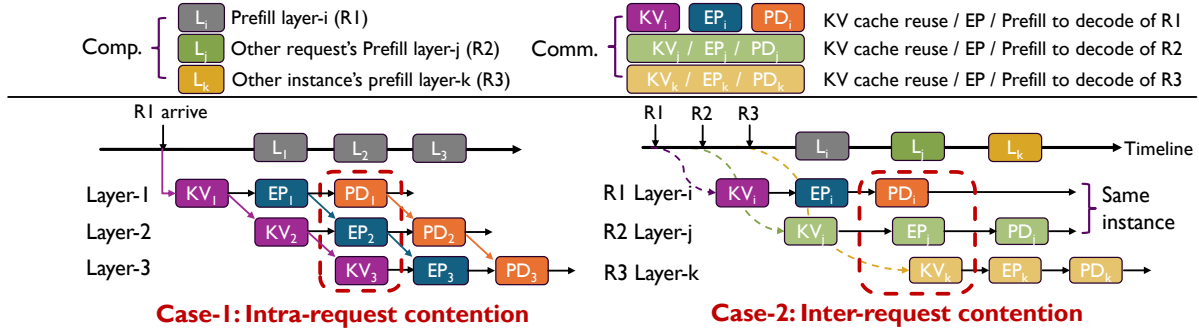


Figure 4: LLM 服务系统中两类主要争用形式示意图。

2.2 多阶段通信中的争用

在现有生产系统中生成第一个 token（图 2）包含多阶段通信：

- **阶段 1: KV-cache 复用**：从远端 prefill 单元拉取可复用的 KV-cache 块；
- **阶段 2: 集合通信**：通过模型并行在设备间交换中间激活；
- **阶段 3: Prefill-to-Decode (P2D) 传输**：将完整的 KV-cache 历史发送到 decode 单元¹。

已有文献指出，单个阶段本身就具有显著开销：集合通信（如 all-to-all 等）会占据端到端时延的 40–60% [39, 40, 92, 98]，而 KV-cache 搬运也占据不可忽视的比例 [19, 42, 51, 70, 94]。本文进一步识别出一个关键但研究不足的问题：集合通信与 KV-cache 搬运在时间上经常重叠，并竞争共享网络带宽。图 3 展示了争用在空间上的位置：KV-cache 传输（阶段 1 和 3）与集合通信（阶段 2）会经过相同的物理互连。这种空间共址造成了两类主要争用，如图 4 所示：

- **请求内争用**：单个请求内部不同通信流之间的相互干扰。具体来说，逐层的 KV-cache 传输（阶段 1 和 3）会与其他层正在进行的集合通信（阶段 2）直接竞争共享链路带宽。
- **请求间争用**：来自不同 prefill 单元的并发请求彼此干扰，导致交换机链路上频繁发生带宽竞争。此外，在块热门度分布倾斜的驱动下，多个 prefill 单元可能同时向同一个远端受害单元拉取“热点”KV 块，从而在该受害单元的本地集合通信（阶段 2）与远程 KV-cache 拉取（阶段 1 和 3）之间，形成对其 NIC 带宽的竞争。

为了量化这类干扰所带来的性能退化，我们在一个 16-GPU 的 prefill 集群（每 GPU 50 Gbps）上进行测量，集群中运行两个服务实例（ $TP = 1, EP = 8$ ）。我们使用 QwenB-agent——一种来自生产环境 Qwen trace [6] 的智能体工作负载，其平均序列长度为 1k token，prompt 复用率为 65%，每 GPU 请求率为 1 req/s；完整测试

¹主流开源服务框架 [63, 74, 85] 会显式地将阶段 3 的时延计入 TTFT 指标。

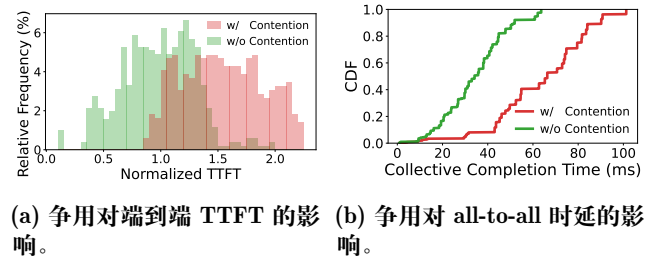


Figure 5: 【测试平台】通信争用对 Mixtral 8x7B 模型的影响。

平台设置见第 6.2 节。我们评估争用对两个指标的影响：端到端 TTFT，以及 All-to-All 操作的集合通信时间（CCT，聚合 Dispatch 与 Combine 两阶段）。具体来说，我们比较理想基线（无争用）与真实服务场景（有争用）。图 5a 展示了 TTFT 分布。结果表明，在存在争用时，整体 TTFT 几乎被拉长了 50%。我们将这一膨胀主要归因于关键路径上 All-to-All 通信时延的恶化：由于与并发 KV-cache 操作发生争用，其 CCT 几乎翻倍（1.8x，见图 5b）。这一显著减速证实，网络争用是 prefill 阶段性能波动的重要来源。

2.3 现有工作的局限

前述分析表明，缺乏管理的多阶段通信争用是 TTFT 违约的主要来源之一。我们将现有方法分为两类：近期的系统级通信优化，以及传统调度算法。如下所述，这两类方法都缺乏对多阶段依赖关系的整体视图，因此难以协调这类争用。

与阶段无关的流调度。 传统数据中心调度策略以流或 coflow 为单位进行管理，优化的目标通常是流/协同流完成时间或截止期限满足率等网络指标。然而，这些目标与端到端应用目标（如 TTFT 达成率）并不一致，因为 TTFT 是由跨多个相互依赖阶段的流共同决定的。因此，这些方法不可避免地会过度优先松弛度较大的流，却让真正用于解锁下游计算的关键传输被饿死，最终恶化 TTFT SLO 达成率。

针对孤立阶段的专用优化。 我们注意到，一些工作分别针对孤立阶段进行优化，具体包括：通过算法综合 [16, 34, 50, 75]、参数调优 [91] 和重叠执行 [46, 98, 103, 104] 来优化集合通信，或者通过流水线预取 [66, 69] 和块合并 [19, 42, 79] 来优化 KV-cache 传输。然而，这些机制通常追求彼此冲突的目标。许多方法通过增加通信并发性来隐藏延迟，但没有一个机制能够在不同通信类型之间统一协调这种并发。结果是，即使引入了这些优化，集合通信和 KV-cache 传输仍常常彼此干扰，反而进一步放大争用。

3 方法

3.1 问题建模

简化抽象。 不失一般性地，我们考虑一个单独的 prefill 请求，它依次经过 L 个顺序 Transformer 层。每一层的通信负载可统一抽象为多阶段流（Multi-stage Flow, MsFlow）。一个 MsFlow 由三个具有时间依赖关系的阶段组成，每个阶段对应由该层生命周期所决定的一组 flow（或 coflow），具体如下：

- **阶段 1：初始化。** 涉及 KV-cache 复用，用于传输先决状态。这类松耦合流往往会与前一阶段计算重叠，以隐藏延迟。
- **阶段 2：执行。** 由集合通信（如 all-to-all）组成，并会严格阻塞后续计算。
- **阶段 3：完成。** 负责把结果通过 P2D 传输到解码 worker。它决定最终 token 的可用时间，但不会阻塞当前 prefill 计算。

对于单个请求而言，所有 MsFlow 共享同一个 TTFT 截止时间，并且会在层依赖约束下发生部分重叠（例如，第 l 层的阶段 2 必须完成后才能开始计算第 $l+1$ 层）。

争用动态。 当并发 MsFlow 竞争共享网络带宽时，就会发生争用。回顾图 4，请求内争用发生在同一请求的不同层 MsFlow 之间，而请求间争用则发生在不同请求的 MsFlow 之间。两者本质上都可归结为异构阶段之间的争用。

总体目标。 我们的目标是满足端到端 TTFT 截止时间（即 SLO 达成），而不是优化单个流时延。具体来说，我们必须联合编排这些存在依赖关系的 MsFlow，以保证全部 L 个 MsFlow 都能在截止约束内完成。为此，理想调度器需要围绕 MsFlow 各阶段的语义，在两个互补目标之间取得平衡：

- 最小化非重叠通信时延。** 对早期阶段（1 和 2）而言，若完成过晚，会直接拉长当前层执行时间，并推迟下一层的释放。
- 最小化提前量（Earliness）。** 对最终阶段 3 而言，过早完成并不会帮助满足截止时间，反而会加剧潜在网络争用。

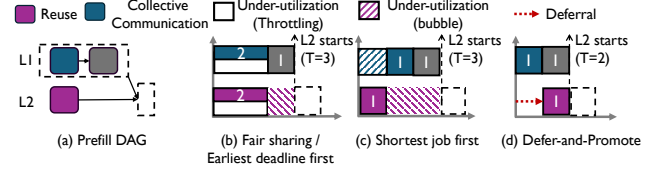


Figure 6: 请求内争用（入口）场景下不同调度策略的对比。 (a) 第 1 层执行期间的入口端口争用。(b)-(c) FS/SJF/EDF 将第 2 层启动时间延迟到 $T = 3$ 。(d) 延后并提升策略将第 2 层启动时间提前到 $T = 2$ （缩短 33%）。

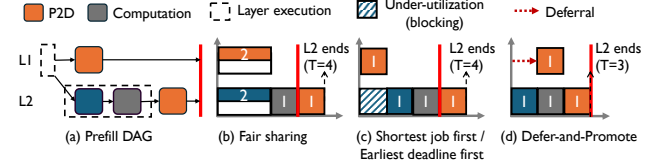


Figure 7: 请求内争用（出口）场景下不同调度策略的对比。 (a) 第 2 层执行期间的出口端口争用。(b)-(c) FS/SJF/EDF 将第 2 层完成时间延迟到 $T = 4$ 。(d) 延后并提升策略将第 2 层结束时间缩短到 $T = 3$ （降低 25%）。

Table 1: 请求间争用示例。

请求 ID	流 ID	流大小	剩余时间	截止时间
1	A	2	9	18
2,3	B	4	6	12
4	C	3	0	7

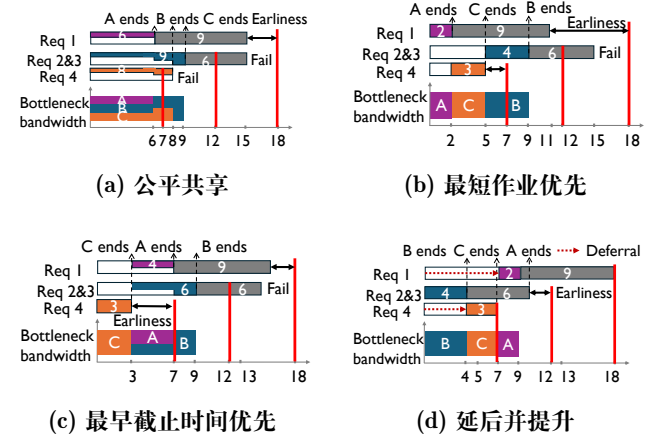


Table 2: 请求间争用场景下不同调度策略的对比。 红色竖线表示硬截止时间，灰色条表示后续持续时间。(a) 公平共享和 (b) SJF 由于缺乏截止时间感知，无法保护紧急的 Flow-B；(c) EDF 会让具有显式但宽松截止时间的流不必要地过早完成；(d) Defer-and-Promote 会有策略地延后非紧急流，以最小化提前量，并仅在必要时提升其优先级，从而实现恰好按时完成。

3.2 关键观察与调度原则

最优解在理论上是难以求得的 [11, 12]。为设计有效启发式方法，我们首先考察 prefill 的执行动态。分析表明，请求截止时间具有一种结构性特征，这启发了本节提出的调度原则。

关键观察。 在 prefill 阶段，随着 prefill 流水线不断推进，请求级截止时间会逐步具体化为流级截止时间。

回顾第 3.1 节，对于初始阶段（1 和 2）而言，流级截止时间是隐式的，因为它由 TTFT 与下游任务时长共同决定。当执行进入最终阶段时，这一约束开始显式化：全局 TTFT 直接决定 token 返回时间，从而转化为明确的流级约束。我们借用实时系统中的“松弛度”（laxity）概念——即触发截止违约前所剩余的预算——来表示流级紧迫性。不同的松弛度揭示出一个调度机会：我们可以安全地延后末阶段流（其松弛度是确定的），从而将带宽让给那些松弛度尚不确定的早期阶段流。

调度原则：Defer-and-Promote。 基于上述观察，我们的核心原则是：将非紧急传输尽可能延后到最后一个安全时刻，仅在必要时提升其优先级，从而在保证按时完成的同时最小化干扰。这一原则可具体化为两条规则：

- **原则 #1（带显式截止时间的末阶段流）** 初始时被延后，以让出带宽；随后逐步提升优先级，以保证满足截止时间。
- **原则 #2（带隐式截止时间的早期阶段流）** 根据相对松弛度进行延后，并随着松弛度减小而提升优先级，以降低违约风险。

为何有效。 通过延后非紧急流并在必要时有选择地提升它们，调度器得以在短期效率与长期截止保证之间取得平衡。这一策略从两个维度提升 TTFT SLO 达成率：(i) 最小化非重叠通信时延，以缩短 prefill 总工期；(ii) 控制提前量，从而在争用条件下优先保障更紧迫的请求。

为理解 Defer-and-Promote 为何能改善 prefill DAG 的工期，考虑图 6-(a)（入口）与图 7-(a)（出口）中的争用场景。Fair Sharing 会不加区分地稀释阶段 2 的吞吐；SJF（图 6-(c)、7-(c)）会优先调度 KV 搬运流（其流大小通常更小）；而 EDF 会因显式截止时间而过度优先阶段 3（图 6-(c)），并在截止时间隐式时退化为 Fair Sharing（图 7-(b)）。因此，这三种策略都会放大非重叠通信时延并阻塞计算。Defer-and-Promote 则通过有策略地延后阶段 3（其松弛度更高）来保护阶段 2，从而显著缩短总工期（图 6-(d)、7-(d)）。

在请求间争用下，Defer-and-Promote 通过严格优先保障真正紧急的请求来最大化达成率。以表 1 为例，每个请求都具备端到端 TTFT 约束，且其剩余时间由下游任务决定。正如图 2 所示，三条具有不同大小和截止时间的流竞争同一条瓶颈链路。与 FS、SJF、EDF 等标准策略经常误判紧迫性不同，Defer-and-Promote 会优先保障集合通信，因为若将其延后，会立刻阻塞执行流水线。与此同时，它会有策略地延后其他流量：阶段

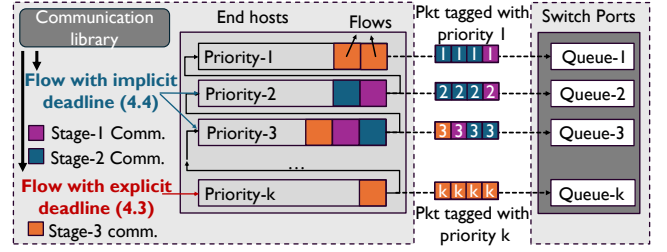


Figure 8: 设计总览。

1 流仅在其即将阻塞流水线时才被提升，而末阶段流则会一直拖延到显式截止时间临近时再执行。

4 设计

4.1 设计挑战

尽管 Defer-and-Promote 原则在概念上相对简单，但在 LLM 服务场景中将其真正落地并不容易。MFS 必须在信息不完备的情况下进行优先级分配：在缺乏精确松弛度的前提下，通过离散多级队列（例如交换机上的有限优先级队列）来引导流。具体而言，我们识别出以下两个挑战。

C-1：如何在不过度优先的前提下调度带显式截止时间的流。 即便某些流携带显式截止时间（如 P2D 传输），在离散优先级层级下，何时从“延后”切换到“提升”仍然并不简单。过早提升会浪费稀缺的瓶颈带宽，而过晚提升则会导致截止违约。难点在于：在有限优先级类别下，如何找到“恰好按时完成”的平衡点。

C-2：如何在没有精确松弛度的情况下调度隐式截止时间流。 缺乏精确松弛度会让调度器陷入容易出错的两难。一方面，若低估松弛度，就会带来意外的执行停顿：误将阶段 2 流延后会立即让 GPU 空转，并拉长请求延迟。另一方面，若高估松弛度，则会造成优先级反转，把稀缺带宽浪费在本可延后的工作上，使真正紧急的请求被饿死。

4.2 总览

图 8 展示了 MFS 的核心抽象：反向多级队列（Reverse Multi-Level Queue, RMLQ）。为了实现 Defer-and-Promote 原则，RMLQ 反转了经典 MLFQ 的思路（后者会随着时间推移降低流优先级）；相反，它会先将流放入低优先级队列以体现“延后”，只有当松弛度收紧、必须立刻执行时，才将其提升。

从高层看，MFS 通过统一的 RMLQ 底座来调度所有多阶段通信，但针对不同阶段采用不同规则（初始优先级与提升速度）。对于带显式截止时间的末阶段流，MFS 采用最小链路利用率（MLU），只有当剩余带宽刚好不足以满足其截止时间时才提升该流，从而避免过早过度优先。对于带隐式截止时间的早期阶段流，MFS 通过相对层索引（Relative Layer Index, RLI）来推测相对紧迫

性。流从较低层级开始，当其可能阻塞计算时再逐步上升。最后，为了解决潜在的跨阶段争用，MFS 将最高优先级保留给带显式截止时间的流，以避免违约；其余优先级层则主要用于带隐式截止时间的流，从而机会性地利用可用带宽。

本节剩余部分组织如下：第 4.3 节介绍带显式截止时间流的调度；第 4.4 节介绍隐式截止时间情形；最后，第 4.5 节将这些组件整合起来，给出完整的 RMLQ 仲裁逻辑。

4.3 显式截止时间下的调度

为应对挑战 C-1，我们需要让末阶段通信任务得到审慎延后——既不过度优先，也不会被饿死——同时严格保证请求截止时间。难点在于，如何将由截止时间导出的连续紧迫度映射到 K 个离散优先级队列上。因此，调度器需要一种机制，用粗粒度优先级近似连续的截止紧迫度，这要求我们回答三个问题：如何量化紧迫度、何时触发提升、以及调度决策应在何种粒度上实施（例如按包还是按层）。

具体地，考虑一个具有 L 层的模型，为一个请求 batch 提供服务。在每一层 ℓ ，P2D 阶段都会产生一组流。同一请求 r 内的流继承其 TTFT 截止时间 D_r ，而不同请求的流独立调度。我们假设系统有 K 个优先级队列 $P_i (1 \leq i \leq K)$ ，其中 P_1 优先级最高。记将优先级从 P_j 提升到 P_{j-1} 的阈值为 $\tau_j (2 \leq j \leq K)$ 。我们定义 $\tau_K = +\infty$ ，以确保即便是极为宽松的流也会落入最低优先级队列。

紧迫度指标。 我们定义最小链路利用率 (Minimal Link Utilization, MLU)： $MLU_i(t) = \frac{Size_{rem}(t)}{Time_{rem}(t) \cdot B \cdot (1-\rho)}$ ，其中 $B(1-\rho)$ 表示扣除背景负载 ρ 后的有效带宽。该指标表示：为了满足截止时间，一个流至少需要占据剩余链路容量的多大份额。关键在于， $MLU_i(t) > 1$ 表示系统进入不可行的过载状态；数值接近 1 则说明已非常紧急，需要几乎独占的服务；较低取值则意味着仍有足够松弛度可供延后。相较之下，仅基于截止时间的指标（如 EDF）并不适合，因为它们会在过载条件下遭遇多米诺效应 [13]，在受限于离散优先级层级时还会进一步退化 [14]。

提升阈值优化。 要推导出能最小化截止违约率的全局最优阈值在计算上是不可行的 (NP-hard)。因此，我们转而寻求一种稳健、实用的近似方法：最小化把连续紧迫度映射到离散层级时引入的相对量化误差。当紧迫度为 $v \in [U_{min}, U_{max}]$ 的流被映射到离散优先级 τ_k 时，会产生相对误差 $\epsilon \approx \frac{|v - \tau_k|}{v}$ 。我们的直觉是最小化所有阈值上的最坏相对误差。由于 $\prod_{k=1}^K r_k = \frac{u_2}{u_1} \cdot \frac{u_3}{u_2} \cdot \dots \cdot \frac{u_K}{u_{K-1}} = \frac{u_K}{u_1} = \frac{U_{max}}{U_{min}}$ ，数学上可知，只有当所有比例都相等 ($r_k \equiv r$) 时，该最坏误差才最小。这要求阈值按几何级数生成（其中 $r = (U_{max}/U_{min})^{\frac{1}{K-1}}$ ）。由于实践中精确的 U_{max} 和 U_{min} 往往未知或动态变化，我们通过经验参数近似这一几何间隔：设置提升阈值为 $Q_i = E^{-i} \cdot U (1 \leq i \leq K-1)$ ，

其中参数 E 与 U 通过经验选取（例如 $E = 4, U = 0.5$ ），以获得稳健表现。

提升行为。 过细粒度的提升决策会在网络调度中带来现实问题。特别是，如果提升在很细粒度上发生（如按包），同一条消息可能被切分到多个优先级队列中。这会导致分组乱序，即后发包超过先发包，从而触发昂贵的传输层恢复，并降低有效吞吐。为避免这一问题，我们将提升限制在层边界上，从而在消息层面保证优先级原子性。关键在于，这种更粗粒度并不会削弱调度表达能力，因为现代 LLM 通常具有很深的网络层数 ($L > K$ ，例如 $L = 64, K = 8/16$)，足以提供丰富的优先级调整空间。此外，我们强制采用单调提升策略（即只允许提升）。这可以避免优先级振荡，并进一步强化 *Defer-and-Promote* 原则：在紧迫性尚未真正达到阈值之前，流会被保守地维持在较低层级。

4.4 隐式截止时间下的调度

为应对隐式截止时间情形下缺乏精确松弛度的问题（挑战 C-2），MFS 利用结构性确定性来推断请求内部的相对紧迫度，并通过稳健仲裁机制来解决请求之间残余的争用。

4.4.1 利用结构确定性的请求内调度。 尽管由于动态调度偏移的存在，精确松弛度难以获得，但单个请求内部的相对松弛度却是确定的：阻塞当前执行的流，必然比只为未来进展做准备的前瞻性传输更紧急。

我们用相对层索引 (RLI) 来量化这一点，其定义为 $RLI = L_{target} - L_{curr}$ ，其中 L_{curr} 是当前正在执行的层号， L_{target} 是该数据被消费的目标层号。较小的 RLI 表示更短的安全延后窗口。MFS 通过严格优先调度 RLI 最小的流来解决争用。下面给出两种代表性争用情形。

情形 I：集合通信与 KV 复用的竞争。 当 $collective^i$ 与 $reuse^j (j > i)$ 竞争时，集合通信对应当前层 ($L_{target} = i$)，因此 $RLI(coll) = 0$ 。相比之下，KV 复用对应第 j 层，故 $RLI(reuse) > 0$ 。因此，我们强制执行 $collective^i \succ reuse^j$ ，以避免执行停顿。

情形 II：KV 复用之间的竞争。 在多层 lookahead 下， $reuse^j$ 与 $reuse^k (L_{curr} < j < k)$ 常常相互竞争。比较其紧迫性时，第 j 层显然对应更近的截止约束，因此有 $RLI(j) < RLI(k)$ 。优先调度 RLI 更小者能够有效推迟最早执行停顿的发生。因此，我们强制执行 $reuse^j \succ reuse^k$ 。

总之，MFS 通过“严格优先最小 RLI”的统一规则来做出上述决策。定理 1 则形式化说明了这一结构性排序的最优性。

定理 1。 在不存在抢占开销的理想计算模型下，严格优先调度 RLI 最小的流，可以最小化 *prefill* 的总工期。

证明思路。 直观地说，在阻塞区间内优先服务较大 RLI 的流，会浪费本可与后续计算重叠的机会。我们通过交换论证证明这一点：若把任意较大 RLI 的流从阻塞区间中移出，并延后到之后的重叠区间执行，则在不破

坏未来依赖关系的前提下，当前阻塞时长会严格缩短。完整证明见附录。

4.4.2 稳健的请求间调度. 当通过 RLI (第 4.4.1 节) 解决了请求内争用后，剩余挑战是：当来自不同请求的竞争流具有相近 RLI，并且同时威胁当前执行时，如何在请求之间打破僵局。调度器需要在松弛度不精确的前提下，优先保护那些 SLO 违约风险更高的请求。然而，同步 batch 执行会放大调度失误的影响，使错误传播到同一 batch 的所有同伴请求。我们的直觉不是恢复精确松弛度，而是让调度决策对估计误差更稳健，从而优先保护那些一旦延迟就会最严重损害 SLO 达成率的请求或 batch。

松弛度不精确下的调度陷阱. 我们识别出两类由于误判松弛度而产生的代表性失败案例：

- **Piggyback 效应.** 当一个 batch 内部同时包含极紧和较松的请求时，少数极端紧急请求会主导推断出的 batch 级紧迫性。结果是，同一 batch 中本来宽松的请求也被一并抬高优先级，从而抢占了全局上更紧急的其他 batch。
- **Black Hole 效应.** 距离截止时间近并不意味着它是可完成的。在过载条件下，一个 batch 看起来可能非常紧急，但其负载量已超过可用带宽。优先调度这类 batch 只会把资源浪费在注定失败的任务上，并阻塞其他本可完成的 batch。

我们定义稳健有效截止时间 (Robust Effective Deadline, RED) 来量化紧迫性，并对抗 Piggyback 效应。具体而言，我们在 batch 内部按最大截止时间间隔将其划分为“紧请求子 batch”和“松请求子 batch”。记 $D_{\min}^T$ 与 $D_{\min}^{Lo}$ 分别为这两个子 batch 的最小截止时间，则定义 $RED = f \cdot D_{\min}^T + (1 - f) \cdot D_{\min}^{Lo}$ ，其中 f 表示紧请求所占比例。当紧请求比例很小 (f 较小) 时，RED 会自然偏向较松的截止时间 ($D_{\min}^{Lo}$)，从而防止少量离群点劫持整个 batch 的优先级。

除基于 RED 的排序之外，我们还采用过载控制来缓解 Black Hole 效应。其直觉在于：避免让注定失败或负担过重的请求阻塞可行请求。我们通过累计估计的计算与通信时延来执行最坏情形可行性检查，并迭代剪除那些对累计时延贡献最大的请求，以恢复剩余 batch 的可行性。

完整算法在附录中给出。它在 batch 到达和离开事件触发时运行，并输出两项结果：(1) 由 RED 决定的有序序列 σ ；(2) 被识别为不可行请求的剪枝集合 $\mathcal{H}$ 。我们刻意避免在每一层都做细粒度更新，以防调度器对瞬时负载估计抖动产生过度反应。

4.5 整体整合

MFS 通过统一的 RMLQ 底座调度所有流量。我们根据阶段特征对流进行分类，并针对初始优先级分配与后续提升规则分别做出设计。

- 对于带显式截止时间的末阶段流，我们计算第 4.3 节定义的 MLU 来确定初始优先级。对于提升操作，MFS 会在请求计算过程中于层边界更新优先级，而在计算结束后则改为固定时间间隔的周期性更新。
- 对于带隐式截止时间的早期阶段流，优先级分配依赖第 4.4.1 节定义的 RLI。阶段 1 流会根据 RLI 进行初始化，并随着计算推进在层边界逐步提升；阶段 2 流则直接进入高优先级队列。

仲裁. MFS 将最高优先级严格保留给最紧急的末阶段流。在其余优先级层中，MFS 采用分层策略：优先让早期阶段流先于末阶段流，以机会性利用可用带宽；而当多个早期阶段流拥有相同 RLI 时，则按照请求间调度 (第 4.4.2 节) 确定的顺序进行裁决。

5 实现

我们用约 10,000 行代码实现了 MFS。MFS 采用双层控制面，由本地守护进程与中心协调器组成。本地守护进程负责维护调度上下文、执行调度决策，并在调度与完成事件发生时向协调器汇报请求级统计信息。协调器以请求粒度工作，聚合这些摘要信息以进行跨节点的平局裁决，并将最终决策回传给本地守护进程执行。

系统集成与兼容性. 我们将 MFS 实现为一个可插拔模块，可透明地与 NCCL 和 Mooncake 集成。每个节点运行一个轻量级本地守护进程，并通过基于共享内存的无锁 MPSC 队列与运行时交互，从而实现低时延的节点内信号传递。通过拦截底层网络原语 (如 RDMA Verbs)，MFS 将库代理线程中的工作请求重定向到本地守护进程，再由后者依据我们的调度逻辑统一编排任务执行。运行时与调度器之间通过三个标准化原语交互：(1) **submit**: 注册带元数据的任务；(2) **permit**: 通过授予特定优先级来控制发送；(3) **completion**: 在硬件完成信号到来时更新内部状态。若请求需要被剪枝，则只需修改其元数据以抑制相应通信。

优先级执行. 商用硬件通常只暴露有限数量的优先级类别 ($K \ll L$)，因此无法直接一一映射。实践中，集合通信通常局限在局部服务单元内部，而 KV-cache 流量则会跨机架穿越整个网络。针对这一特点，我们设计了混合优先级队列。具体而言，MFS 在主机侧通过软件队列执行严格优先级 (仲裁逻辑见第 4.5 节)，同时利用基于 DSCP 的硬件优先级，在网络中隔离 KV-cache 流量。对于带显式截止时间的流，我们根据 K 与当前网络负载 ρ 计算 MLU 阈值，并据此分配 DSCP 标记；对于带隐式截止时间的流，我们将其 RLI 映射到物理队列区间 $[0, K - 1]$ ，方式是将 RLI 截断在 $K - 1$ 以内。

6 评估

6.1 实验设置

测试平台设置. 测试平台由 8 台服务器组成，每台服务器配备 4 块 NVIDIA GeForce RTX 3090 GPU、40 个 CPU 核 (Intel(R) Xeon(R) Gold 5218R CPU @

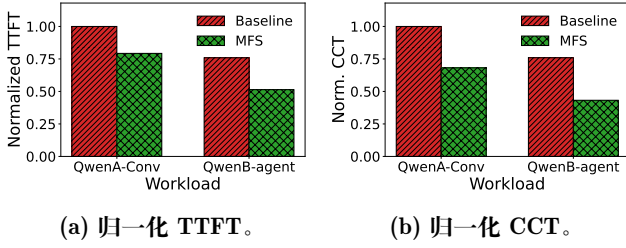


Figure 9: 【测试平台】Mixtral-8x7B

2.10GHz)、128GB 内存以及两张 ConnectX-5 100Gbps NIC。服务器通过 NVIDIA SN2700 以太网交换机互连。单机内部 GPU 通过 PCIe Gen 3.0 通信。所有服务器运行 Ubuntu 22.04、CUDA 12.8、PyTorch 2.8.0 和 NCCL 2.28。我们配置了一个由 16 块 GPU 组成的 prefill 集群，并单独配置 decode worker。为简化分析，我们将 decode 输出长度限制为 1 个 token，因为这不会影响 TTFT。

仿真设置。 我们的模拟器构建在 Vidur [5] 与流级模拟器 flowsim [82] 之上。我们扩展了 Vidur，使其支持带专家并行和序列并行的 Mixtral 模型，以及带 KV-cache 复用的 PD 解耦。仿真流程首先通过 Vidur profiler 离线剖析算子时延；运行时，我们采用统一的事件驱动机制：Vidur 负责模拟 LLM 服务行为并生成计算与通信任务，而 flowsim 负责解析通信任务并触发底层网络事件。关键在于，计算事件与网络事件都在同一个事件队列中处理，以保证正确性。默认情况下，我们模拟一个 256 台服务器的集群，prefill 与 decode worker 比例为 1:1，并由一个类似 Dynamo [62] 的 KV-cache 感知调度器统一管理。每台服务器包含 8 块通过 NVSwitch (900 GB/s) 互连的 GPU 和 8 张 NIC，整个系统采用 1:1 fat-tree 网络，链路带宽为 200Gbps。时延剖面基于 NVIDIA A100 GPU 标定。

指标。 我们主要评估首 Token 时间 (TTFT) 及其服务等级目标 (SLO) 达成率。遵循 [29, 67, 78] 的方法，默认将 SLO 阈值定义为低负载条件下 TTFT 的 3 $\times$ 。我们也评估了一些微基准指标，例如集合通信完成时间 (CCT) 和请求提前量 (earliness)，以帮助解释性能收益来源。

6.2 测试平台实验

模型与工作负载。 在测试平台实验中，我们评估 Mistral 8x7B [3] MoE 模型 (FP16, top-k=2)，其张量并行度 (TP) 设置为 1，专家并行度 (EP) 设置为 8。工作负载方面，我们采用两条来自 Qwen 的生产轨迹 [6]。其中，QwenA-Conv 表示对话型工作负载，平均序列长度为 2k token，prompt 复用率为 50%；QwenB-agent 表示智能体型工作负载，平均长度为 1k token，复用率更高，为 65%。请求到达遵循泊松过程，我们通过调整每秒请求数 (RPS) 来控制系统负载。受测试平台容量限

制，我们将每条轨迹的前 512 个请求作为预热阶段，并使用之后的 1024 个请求进行评估。

图 9 展示了 MFS 在 QwenA-Conv 与 QwenB-Agent 两类工作负载上的表现。我们将 vLLM 的 batched tokens 设置为 8192，并将每 GPU 请求率设为每秒 1 个请求。MFS 在两种工作负载下都显著降低了 TTFT：在 QwenA-Conv 上，平均 TTFT 优化 20.7% (1.26 $\times$)；在 QwenB-Agent 上，优化 32.3% (1.48 $\times$)。TTFT 的下降主要来自于阻塞计算的集合通信更快完成。以 all-to-all 完成时间 (CCT) 为例，MFS 在 QwenA-Conv 上将阶段 2 通信缩短 31.9% (1.47 $\times$)，在 QwenB-Agent 上缩短 43.1% (1.76 $\times$)。这些集合通信处于 prefill 执行的关键路径上：一旦被拖慢，就会立即阻塞 GPU 计算并抬高端到端延迟。

6.3 大规模仿真

在大规模仿真中，我们扩大了评估范围，以覆盖多种生产级架构与工作负载。

- 我们考察了一组主流的专家并行 MoE 模型：既包括专家规模大但数量较少的架构（如 Mixtral 8x22B [2] 和 Grok-2 [1]，配置为 TP=4, EP=8），也包括专家更小但数量更多的模型（如 DBRX [23]，配置为 TP=2, EP=16，以及 Qwen3-Coder [71]，配置为 TP=1, EP=32）。这些模型均部署在 32-GPU 实例上，并采用混合并行策略。我们利用 Qwen 对话轨迹与智能体轨迹来代表典型交互行为。
- 我们还进一步评估 MFS 在采用序列并行 (SP) 的稠密模型上的有效性 [48]。我们评估了配置为 1M token 上下文窗口的 Llama [54] 模型。在该设置下，模型部署于 16-GPU 实例上，采用 TP=4, SP=4。我们将其与 Mooncake 数据集 [70] 配对，以模拟具有相似复用率（约 ~40%、~65%）和更长上下文长度（约 ~15k、~9k）的真实对话和智能体工作负载。

基线。 我们将 MFS 与四类经典流调度策略进行比较：

- **Fair Sharing**：对并发流实施 max-min 公平，按等带宽分配，不考虑流大小与截止时间。
- **Shortest Job First (SJF)**：严格优先调度更小的流，以最小化平均流完成时间。
- **Earliest Deadline First (EDF)**：优先调度带显式截止时间的流。由于应用级截止时间无法直接映射为单个网络流截止时间，对带隐式截止时间的流则退化为公平共享。
- **Karuna [17]**：为带截止时间的流分配确保按时完成所需的最小带宽，而剩余流量按 SJF 调度。

端到端 TTFT SLO 达成率。 我们在不同请求率、不同模型与工作负载下评估端到端 TTFT SLO 达成率。结果如图 10、11 和 12 所示，MFS 在所有场景中都稳定优于当前最先进的基线方案。

图 10 展示了 MFS 在多种 MoE 模型上的对话工作负载表现。具体来说，在高请求率下，MFS 在 Mixtral

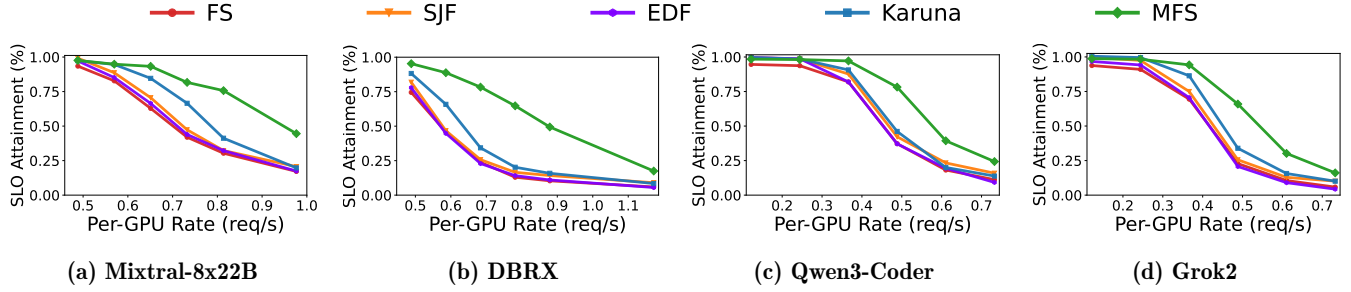


Figure 10: 【仿真】对话工作负载 (QwenA) 上的性能

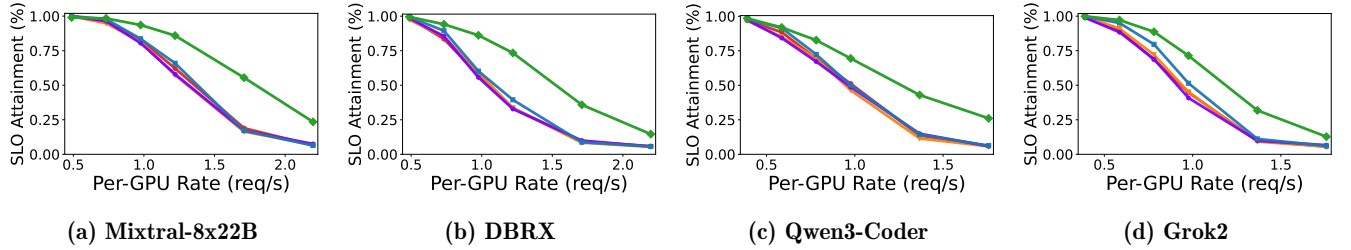


Figure 11: 【仿真】智能体工作负载 (QwenB) 上的性能

(图 10a)、Qwen Coder (图 10c) 和 Grok (图 10d) 上的 SLO 达成率提升为 $1.4\times$ – $1.8\times$ ，而在 DBRX (图 10b) 上最高达到 $2.4\times$ 。此外，在维持相近 SLO 达成率的前提下，MFS 能承受比最强基线 Karuna 高出 $1.17\times$ – $1.46\times$ 的请求率。在对话工作负载中，一小部分尾部请求需要大规模 KV-cache 迁移，这会在高负载下触发从阶段 1 到阶段 3 的跨阶段争用。MFS 能显著优于基线，正是因为它成功协调了多阶段争用。在各基线中，Karuna 相对表现较好，因为它碰巧会延后非紧急的阶段 3 流量，从而提供一定缓解；但由于缺乏显式的多阶段协同，它最终仍会像其他基线一样被争用拖垮。

图 11 展示了 MFS 在多种 MoE 模型上的智能体工作负载表现。聚焦中高负载区间，MFS 在 Grok-2 与 Mixtral 上实现 $1.4\times$ – $1.7\times$ 更高的 SLO 达成率，在 DBRX 和 Qwen-Coder 上可达到 $2.0\times$ 。此外，与 Karuna 相比，MFS 可支持 Grok-2 高出 $1.2\times$ 的请求率，Mixtral 和 Qwen-Coder 约高出 $\sim 1.3\times$ ，DBRX 则可达 $1.4\times$ 。相较于对话轨迹，智能体请求的 prompt 更短，但平均复用率更高，多个并发请求经常共享完全相同的前缀。这种模式诱发了严重的一对多发送端争用，其主要集中在阶段 1 和阶段 2，因为多个 worker 会同时争抢同一批缓存状态。MFS 无需精确松弛度估计，就能通过保护紧急流有效缓解这种争用。值得注意的是，在该场景下，Karuna 与其他基线之间的差距有所缩小。当争用并非主要由阶段 3 流量主导时，Karuna 会退化为使用 SJF 来调度阶段 1 和阶段 2 流（因为截止时间是隐式的）；然而，由于其不理解阶段语义，这种做法仍然会像其他基线一样，让紧迫请求暴露于 SLO 违约风险之下。

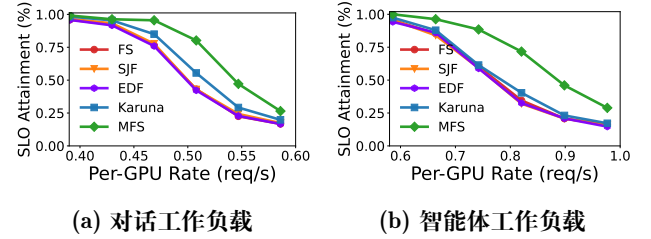


Figure 12: 【仿真】Llama3-8B (Mooncake 数据集)

图 12 展示了 MFS 在采用序列并行 (SP) 的 Llama3-8B 模型上、两个代表性长上下文工作负载中的表现。对于 Mooncake 数据集，对话与智能体工作负载在请求模式上与 Qwen 实验相似（例如复用率、访问模式），但上下文长度显著更长。在对话工作负载中（图 12a），MFS 在负载上升时依然表现稳健，相比 Karuna 实现了 $1.3\times$ – $1.6\times$ 的 SLO 达成率提升。这一优势在智能体工作负载中进一步放大（图 12b）：MFS 实现了 $1.4\times$ – $1.9\times$ 的更高 SLO 达成率，并在严格 SLO 预算下支持最高 $1.15\times$ 的每 GPU 请求率提升。

性能收益拆解。 我们进一步设计了微基准实验，以分析收益来源。正如图 13 与图 14 所示，MFS 成功实现了两点：(i) 最小化集合通信完成时间（即非重叠通信时延），从而缩短 prefill 总工期；(ii) 调控提前量，在争用条件下优先保障真正紧急的请求。

图 13 对 MFS 在 DBRX 模型上实现 $2.2\times$ 更高 SLO 达成率的收益进行了拆解。第一项收益来自 prefill 阶段执行时间的缩短。具体而言，MFS 将专家并行的平均 CCT 降低了 52%（图 13a）。这一收益主要来源于

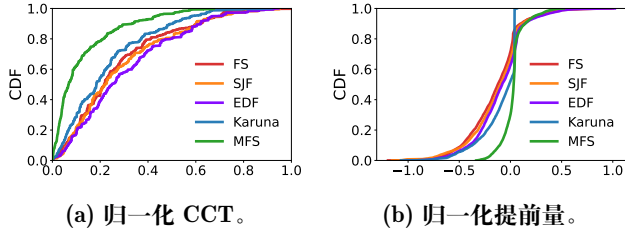


Figure 13: 【仿真】DBRX 上的收益拆解 (Qwen-A, Rate=0.7)

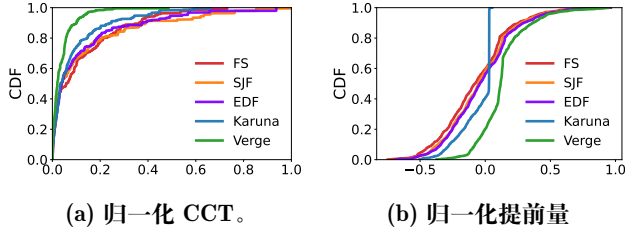


Figure 14: 【仿真】Llama3 上的收益拆解 (Mooncake-Conv, Rate=0.5)

我们的阶段感知策略：它优先保护会阻塞计算的流，同时延后非关键流量，以利用可用松弛度实现重叠。因此，MFS 通过降低有效通信开销，显著缩短了 prefill 总工期。

除了降低时延外，MFS 还通过更准确地优先真正紧急的请求，进一步提升了 SLO 达成率。我们使用请求的提前量 (earliness) 来刻画这一现象：较大的正值意味着不必要的过早完成，这会阻塞更紧急的请求；负值则对应截止违约。如图 13b 所示，Karuna 的非负提前量最小，但其代价是更高的违约风险，因为它采用近乎最小速率的保守分配方式，无法有效利用可用带宽。相比之下，其他基线表现出更大的非负提前量，说明它们在调度流时没有有效体现紧迫性。与 FS、SJF、EDF 相比，MFS 将非负部分的提前量降低了 42%。通过把阶段 3 流量延后到恰好按时完成，并利用基于 RED 的排序来调控早期阶段流量，MFS 为真正紧急的请求保留了资源。MFS 同时采用基于优先级的调度方式，因此在没有更高优先级流等待时，仍可充分利用可用带宽。

我们在图 14 中观察到了相似趋势。在这两组微基准中，MFS 都通过同时降低序列并行的集合通信完成时间 (图 14a) 并调控争用下的提前量 (图 14b)，稳定实现更高的 SLO 达成率。这些结果表明，MFS 的性能收益在不同模型间具有稳健性，并不依赖某一种特定工作负载或架构。

7 讨论

更多并行方式。 尽管 MFS 主要关注专家并行与序列并行，我们也意识到仍存在一些未在评估中显式覆盖的并行策略。张量并行虽然在服务系统中被广泛采用，但

通常局限于单机内互连 (如 NVLink)，不会直接与跨节点带宽发生竞争。类似地，我们也没有显式讨论流水线并行，因为 prefill 阶段通常对时延极其敏感 (因此要求较低甚至为零的 PP 度数 [106])，而其通信量相较于 KV cache 搬运或集合通信要小几个数量级 [44]。如果这类流确实出现在共享网络上，它们会被识别为阶段 2 流量，因此仍与我们的分析兼容。这一观察也适用于新兴并行策略：尽管它们可能引入新的通信模式和更强的重叠能力，但一旦延迟仍会阻塞依赖计算。

混合模型部署。 在真实部署中，服务提供者往往维护分层的模型组合：一个大型旗舰模型配合多个较小模型，共同服务多样化工作负载 [65]。在这种环境下，网络争用主要来自不同模型之间并发的 KV cache 传输 (阶段 1 和阶段 3)，因为集合通信通常被限制在隔离的机架或专用 pod 内部 [46]。虽然本文未显式评估多模型部署场景，但 MFS 的调度原则仍适用于共享网络上的 KV cache 流量。这一方向很有前景，但如何整体协调多模型争用格局，仍是一个研究尚不充分的设计空间。

对 scale-up 互连的适用性。 虽然 MFS 聚焦于 scale-out 网络，其设计原则同样可以扩展到 scale-up 领域 (如 NVL [58]、UB [108] 等)。当多个服务实例共同部署时，集合通信与 KV Cache 搬运之间的争用依然存在。MFS 的调度策略可以借助硬件级隔离机制 (如虚拟通道或流量类别) 有效迁移到这类架构上，从而实现差异化优先控制。

与 GPU 发起式集合通信的兼容性。 尽管新兴的 GPU 发起式通信 [24, 64] 能够优化 decode 阶段的小消息时延，但由于硬件前提严格、对大体量传输收益有限，它在 prefill 部署中仍不常见。即便在这类环境中，MFS 仍可通过 CPU 辅助变体 (如主机侧 doorbell) 保持兼容；尽管相较完全 GPU 发起执行，这种方式可能引入可忽略的额外时延 [68]。

8 相关工作

流与协同流调度。 传统流调度主要优化平均流完成时间 [7, 8, 10, 28, 41, 43, 107] 或截止满足率 [28, 83, 88, 96] 等单流指标，对应策略包括 SJF 和 EDF。Coflow 调度 [4, 21, 22, 80, 86, 97, 100] 进一步把相关流组成一个整体，以优化数据并行工作负载中的 Coflow Completion Time；而混合流调度 [17] 则考虑截止敏感流量与尽力而为流量共存的情形。与之相比，MFS 显式刻画多阶段依赖关系，并以端到端 TTFT 达成为目标进行流调度。

针对单个阶段的优化。 既有研究多半聚焦于孤立通信阶段的优化。对于集合通信，已有工作通过算法综合 [16, 75]、解决潜在争用 [15, 72]、或利用系统级流水线增强重叠 [30, 98, 103] 来提升性能。与此同时，另一些工作则通过预取隐藏时延 [66, 69]、块合并 [19, 42] 与多路径传输 [90] 来优化 KV cache 传输，或者通过牺牲模型质量来降低传输数据量 [51]。然而，这些方法都将

各个阶段孤立处理，忽视了集合通信与 KV cache 传输之间的潜在争用。

LLM 服务系统。 近年来，LLM 服务系统的发展主要沿两条互补路径展开。从架构上看，系统采用 prefill-decode 解耦 [66, 106] 与 KV-cache 复用 [26, 69] 来提升吞吐并避免重复计算。从编排角度看，大量请求调度器 [18, 29, 33, 78, 99] 被提出，用以最大化 SLO 达成率。然而，这些系统主要聚焦于理想网络假设下的计算效率优化。MFS 与这些工作是正交的：它填补了它们所忽视的网络争用问题。

9 结论

本文提出了 MFS，一个面向 LLM 服务的整体式多阶段流调度框架，目标是提升端到端 TTFT SLO 达成率。MFS 建立在这样一个观察之上：随着 prefill 执行推进，请求级截止时间会逐步具体化为显式的流级截止时间。其核心是实现了一种 Defer-and-Promote 调度原则，在无需精确估计松弛度的情况下近似 Least-Laxity-First 行为。通过原型实现与广泛评估，我们表明 MFS 能够在多种模型与工作负载下稳定提升 TTFT SLO 达成率。

REFERENCES

- [1] [n. d.]. Grok. <https://x.ai/news/grok-2>. ([n. d.]).
- [2] [n. d.]. Mixtral 8x22B. <https://huggingface.co/mistralai/Mixtral-8x22B-Instruct-v0.1>. ([n. d.]).
- [3] [n. d.]. Mixtral-8x7B-Instruct-v0.1. <https://huggingface.co/mistralai/Mixtral-8x7B-Instruct-v0.1>. ([n. d.]).
- [4] Saksham Agarwal, Shijin Rajakrishnan, Akshay Narayan, Rachit Agarwal, David Shmoys, and Amin Vahdat. 2018. Sincronia: Near-optimal network design for coflows. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 16–29.
- [5] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.
- [6] Alibaba-Edu. 2025. Qwen-Bailian Anonymous Dataset: Production-derived LLM Usage Traces. <https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon>. (2025). Accessed: 2026-02-06.
- [7] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.
- [8] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: Minimal near-optimal datacenter transport. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 435–446.
- [9] Reza Yazdani Aminabadi, Samyam Rajbhandari, Amar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [10] Wei Bai, Li Chen, Kai Chen, Dongsu Han, Chen Tian, and Hao Wang. 2015. {Information-Agnostic} flow scheduling for commodity data centers. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 455–468.
- [11] Riccardo Bettati and Jane WS Liu. 1990. Algorithms for end-to-end scheduling to meet deadlines. In *Proceedings of the Second IEEE Symposium on Parallel and Distributed Processing 1990*. IEEE, 62–67.
- [12] Riccardo Bettati and Jane W-S Liu. 1992. End-to-End Scheduling to Meet Deadlines in Distributed Systems.. In *ICDCS*. 452–459.
- [13] Giorgio Buttazzo, Marco Spuri, and Fabrizio Sensini. 1995. Value vs. deadline scheduling in overload conditions. In *Proceedings 16th IEEE Real-Time Systems Symposium*. IEEE, 90–99.
- [14] Giorgio C Buttazzo. 2005. Rate monotonic vs. EDF: Judgment day. *Real-Time Systems* 29, 1 (2005), 5–26.
- [15] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.
- [16] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, et al. 2025. SyCCL: Exploiting Symmetry for Efficient Collective Communication Scheduling. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 645–662.
- [17] Li Chen, Kai Chen, Wei Bai, and Mohammad Alizadeh. 2016. Scheduling mix-flows in commodity datacenters with karuna. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 174–187.
- [18] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B Gibbons. 2025. SLOs-Serve: Optimized Serving of Multi-SLO LLMs. *arXiv preprint arXiv:2504.08784* (2025).
- [19] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. 2024. KVDirect: Distributed Disaggregated LLM Inference. *arXiv preprint arXiv:2501.14743* (2024).
- [20] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023) 2, 3 (2023), 6.
- [21] Mosharaf Chowdhury and Ion Stoica. 2015. Efficient coflow scheduling without prior knowledge. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 393–406.
- [22] Mosharaf Chowdhury, Yuan Zhong, and Ion Stoica. 2014. Efficient coflow scheduling with varys. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 443–454.
- [23] Databricks. [n. d.]. Introducing DBRX: A New State-of-the-Art Open LLM. <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>. ([n. d.]).
- [24] DeepSeek-AI. 2024. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>. (2024). GitHub repository, Accessed: 2026-02-07.

- [25] Vignesh Ethiraj, Ashwath David, Sidhanth Menon, and Divya Vijay. 2025. Toward Low-Latency End-to-End Voice Agents for Telecommunications Using Streaming ASR, Quantized LLMs, and Real-Time TTS. *arXiv preprint arXiv:2508.04721* (2025).
- [26] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [27] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
- [28] Chi-Yao Hong, Matthew Caesar, and P Brighten Godfrey. 2012. Finishing flows quickly with preemptive scheduling. *ACM SIGCOMM Computer Communication Review* 42, 4 (2012), 127–138.
- [29] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. [n. d.]. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Eighth Conference on Machine Learning and Systems*.
- [30] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, et al. 2025. FlashOverlap: A Lightweight Design for Efficiently Overlapping Communication and Computation. *arXiv preprint arXiv:2504.19519* (2025).
- [31] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. 2024. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565* (2024).
- [32] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181* (2024).
- [33] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Taiyi Wang, Bin Cui, Ana Klimovic, and Eiko Yoneki. 2025. Thunderserve: High-performance and cost-efficient llm serving in cloud environments. *arXiv preprint arXiv:2502.09334* (2025).
- [34] Heehoon Kim, Junyeol Ryu, and Jaejin Lee. 2024. Tccl: Discovering better communication paths for pcie gpu clusters. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 999–1015.
- [35] Kaeun Kim, Ghazal Shams, and Kawon Kim. 2025. From Seconds to Sentiments: Differential Effects of Chatbot Response Latency on Customer Evaluations. *International Journal of Human-Computer Interaction* (2025), 1–17.
- [36] kimi. 2025. mooncake transfer engine. <https://github.com/kvccache-ai/Mooncake/tree/main/mooncake-transfer-engine>. (2025).
- [37] Langfuse. 2025. How to configure retries and timeouts when fetching prompts? <https://langfuse.com/faq/all/error-handling-and-timeouts>. (2025).
- [38] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
- [39] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 945–959.
- [40] Jialong Li, Shreyansh Tripathi, Lakshay Rastogi, Yiming Lei, Rui Pan, and Yiting Xia. 2025. Optimizing Mixture-of-Experts Inference Time via Model Deployment and Communication Scheduling. *IEEE Transactions on Networking* 34 (2025), 2478–2497.
- [41] Wenxin Li, Xin He, Yuan Liu, Keqiu Li, Kai Chen, Zhao Ge, Zewei Guan, Heng Qi, Song Zhang, and Guyue Liu. 2024. Flow scheduling with imprecise knowledge. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 95–111.
- [42] Weiqing Li, Guochao Jiang, Xiangyong Ding, Zhangcheng Tao, Chuzhan Hao, Chenfeng Xu, Yuewei Zhang, and Hao Wang. 2025. FlowKV: A Disaggregated Inference Framework with Low-Latency KV Cache Transfer and Load-Aware Scheduling. *arXiv preprint arXiv:2504.03775* (2025).
- [43] Ziyang Li, Wei Bai, Kai Chen, Dongsu Han, Yiming Zhang, Dongsheng Li, and Hongfang Yu. 2017. Rate-aware flow scheduling for commodity data center networks. In *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*. IEEE, 1–9.
- [44] Xudong Liao, Yijun Sun, Han Tian, Xinchun Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, et al. 2025. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 554–574.
- [45] LiteLLM. 2025. Stream-timeout. <https://docs.litellm.ai/docs/proxy/timeout>. (2025).
- [46] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [47] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).
- [48] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).
- [49] Kaiyuan Liu, Xiaobo Zhou, and Li Li. 2025. m2LLM: A Multi-Dimensional Optimization Framework for LLM Inference on Mobile Devices. *IEEE Transactions on Parallel and Distributed Systems* (2025).
- [50] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 16–37.
- [51] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.

- [52] Medium. 2025. Robust LLM API Strategies. <https://ai.gopubby.com/robust-llm-api-strategies-retries-fallbacks-in-python-caf9efa96908>. (2025).
- [53] Medium. 2025. Stream API and timeouts. <https://diverger.medium.com/speeding-up-your-openai-llm-applications-f0e011f2f0d6>. (2025).
- [54] Meta. 2025. llama. <https://huggingface.co/meta-llama>. (2025).
- [55] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [56] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.
- [57] notdiamond.ai. 2025. fallbacks-and-timeout. <https://docs.notdiamond.ai/docs/fallbacks-and-timeouts>. (2025).
- [58] NVIDIA. 2024. NVIDIA GB200 NVL72: A Revolutionary Liquid-Cooled Rack Scale Solution. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>. (2024). Accessed: 2026-02-07.
- [59] NVIDIA. 2025. GeForce RTX 3090 Family. <https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3090-3090ti/>. (2025).
- [60] NVIDIA. 2025. NVIDIA Collective Communication Library (NCCL). <https://github.com/NVIDIA/nccl>. (2025).
- [61] NVIDIA. 2025. NVIDIA Mellanox ConnectX-5. <https://www.nvidia.cn/networking/ethernet/connectx-5/>. (2025).
- [62] NVIDIA. 2026. Dynamo: A Datacenter Scale Distributed Inference Serving Framework. <https://github.com/ai-dynamo/dynamo>. (2026). Accessed: 2026-02-06.
- [63] NVIDIA and Dynamo Authors. 2025. Disaggregated Serving Design Document. https://github.com/ai-dynamo/dynamo/blob/main/docs/design_docs/disagg_serving.md. (2025). Accessed: 2026-02-06.
- [64] NVIDIA Corporation. 2024. *NVSHMEM Documentation: Using the NVSHMEM InfiniBand GPUDirect Async Transport*. NVIDIA Corporation. <https://docs.nvidia.com/nvshmem/api/using.html#using-the-nvshmem-infiniband-gpudirect-async-transport> Release 3.0.6, Accessed: 2026-02-07.
- [65] OpenAI. 2025. ChatGPT. <https://openai.com/index/chatgpt/>. (2025).
- [66] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [67] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. Queue management for slo-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 18–35.
- [68] Sreeram Potluri, Pak Markthub, and Manjunath Gorentla Venkata. 2024. Enhancing Application Portability and Compatibility Across New Platforms Using NVIDIA Magnum IO NVSHMEM 3.0. <https://developer.nvidia.com/blog/enhancing-application-portability-and-compatibility-across-new-platforms-using-nvidia-magnum-io-nvshmem-3-0/>. (May 2024). Accessed: 2026-02-07.
- [69] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.
- [70] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kv-cache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079* (2024).
- [71] Qwen Team. [n. d.]. *Qwen3-Coder-Next Technical Report*. Technical Report. https://github.com/QwenLM/Qwen3-Coder/blob/main/qwen3_coder_next_tech_report.pdf Accessed: 2026-02-03.
- [72] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.
- [73] Bhala Ranganathan, Mickey Zhang, and Kai Wu. 2025. Enhancing reliability in AI inference services: An empirical study on real production incidents. *arXiv preprint arXiv:2511.07424* (2025).
- [74] SGLang Team. 2025. P/D Disaggregation Optimization. https://docs.sglang.ai/advanced_features/pd_disaggregation.html. (2025). Accessed: 2026-02-06.
- [75] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 593–612.
- [76] Haiying Shen and Tanmoy Sen. 2025. AccelGen: Heterogeneous SLO-Guaranteed High-Throughput LLM Inference Serving for Diverse Applications. *arXiv preprint arXiv:2503.13737* (2025).
- [77] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Jingliang Ren, et al. 2025. Collective Communication for 100k+ GPUs. *arXiv preprint arXiv:2510.20171* (2025).
- [78] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
- [79] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. D\`ej\`avu: Kv-cache streaming for fast, fault-tolerant generative llm serving. *arXiv preprint arXiv:2403.01876* (2024).
- [80] Hengky Susanto, Hao Jin, and Kai Chen. 2016. Stream: Decentralized opportunistic inter-coflow scheduling for datacenter networks. In *2016 IEEE 24th International Conference on Network Protocols (ICNP)*. IEEE, 1–10.

- [81] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314* (2023).
- [82] TL-System. 2024. FlowSim. <https://github.com/TL-System/FlowSim>. (2024). Accessed: 2024-05-20.
- [83] Balajee Vamanan, Jahangir Hasan, and TN Vijaykumar. 2012. Deadline-aware datacenter tcp (d2tcp). *ACM SIGCOMM Computer Communication Review* 42, 4 (2012), 115–126.
- [84] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. 2025. Aqua: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 48–62.
- [85] vLLM Team. 2025. Prefill/ Decode Disaggregation in vLLM. https://docs.vllm.ai/en/stable/features/disagg_prefill.html. (2025). Accessed: 2026-02-06.
- [86] Xinchun Wan, Xinyu Yang, Kaiqiang Xu, Xudong Liao, Yilun Jin, Yijun Sun, Zhenghang Ren, Han Tian, and Kai Chen. 2025. Coflow Scheduling for LLM Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1232–1234.
- [87] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Rong Gu, Nguyen Cam-Tu, Chen Tian, and Sheng Zhong. 2024. Revisiting slo and goodput metrics in llm serving. *arXiv preprint arXiv:2410.14257* (2024).
- [88] Christo Wilson, Hitesh Ballani, Thomas Karagiannis, and Ant Rowtron. 2011. Better never than late: Meeting deadlines in datacenter networks. *ACM SIGCOMM Computer Communication Review* 41, 4 (2011), 50–61.
- [89] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654.
- [90] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilun Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, et al. 2026. DualPath: Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference. *arXiv preprint arXiv:2602.21548* (2026).
- [91] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. {AutoCCL}: Automated Collective Communication Tuning for Accelerating Distributed and Parallel {DNN} Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 667–683.
- [92] Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. 2024. Context parallelism for scalable million-token inference. *arXiv preprint arXiv:2411.01783* (2024).
- [93] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [94] Dongha Yoon, Younghoon Min, Hoshik Kim, Sam H Noh, and Jongryool Kim. 2025. TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale. *arXiv preprint arXiv:2512.18194* (2025).
- [95] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan Suchow, Zhenyu Cui, Rong Liu, et al. 2024. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems* 37 (2024), 137010–137045.
- [96] Hong Zhang, Kai Chen, Wei Bai, Dongsu Han, Chen Tian, Hao Wang, Haibing Guan, and Ming Zhang. 2015. Guaranteeing deadlines for inter-datacenter transfers. In *Proceedings of the Tenth European Conference on Computer Systems*. 1–14.
- [97] Hong Zhang, Li Chen, Bairen Yi, Kai Chen, Mosharraf Chowdhury, and Yanhui Geng. 2016. CODA: Toward automatically identifying and scheduling coflows in the dark. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 160–173.
- [98] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, et al. 2025. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *arXiv preprint arXiv:2502.19811* (2025).
- [99] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. In *arXiv preprint arXiv:2504.20068*.
- [100] Yangming Zhao, Kai Chen, Wei Bai, Minlan Yu, Chen Tian, Yanhui Geng, Yiming Zhang, Dan Li, and Sheng Wang. 2015. Rapier: Integrating routing and scheduling for coflow-aware data center networks. In *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 424–432.
- [101] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [102] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [103] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, et al. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv preprint arXiv:2504.19442* (2025).
- [104] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, et al. 2025. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives. *arXiv preprint arXiv:2503.20313* (2025).
- [105] Li Zhong and Zilong Wang. 2024. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 21841–21849.
- [106] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

- [107] Yibo Zhu, Hagga Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion control for large-scale RDMA deployments. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 523–536.
- [108] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. 2025. Serving Large Language Models on Huawei CloudMatrix384. *arXiv preprint arXiv:2506.12708* (2025).

A 定理 1 的证明

本节给出定理 1 中 *Smallest-RLI-First* 策略最优性的形式化证明。

A.1 问题设置与定义

我们将 prefill 过程建模为跨层交替出现的通信与计算阶段序列，层编号为 $\ell = 1, \dots, N$ 。记 $L_{\text{curr}}(t)$ 为时刻 t 正在计算或即将计算的层号。对任意通信流 f ，记 $L_{\text{target}}(f)$ 为其数据被消费的目标层。相对层索引定义为 $\text{RLI}(f, t) = L_{\text{target}}(f) - L_{\text{curr}}(t)$ 。我们采用定理中的假设：

- (i) **理想计算**：一旦某层 ℓ 的依赖（即所有满足 $L_{\text{target}} = \ell$ 的流）被满足，该层计算便立即开始，并连续运行 C_ℓ 时长；
- (ii) **流体模型**：通信流可无限细分，因此可无额外开销地进行抢占；
- (iii) **专用带宽**：我们聚焦单一链路约束（例如入口或出口瓶颈）。

A.2 最优性证明

目标。最小化 prefill 总工期，等价于最小化最终计算层的完成时间。由于总计算量固定，这又等价于最小化执行停顿（即 GPU 因等待数据而空闲的区间）的总时长。

区间分类。我们依据系统状态，将时间线 $[0, T]$ 划分为两类区间：

- **停顿区间 (I_{stall})**：GPU 因存在某个尚未完成且满足 $\text{RLI}(f, t) = 0$ （即目标即为当前层）的流而空闲等待；
- **重叠区间 (I_{overlap})**：GPU 正在执行计算。在这些区间中，链路可用于传输满足 $\text{RLI}(f, t) > 0$ 的流（即未来层所需数据）。

交换论证。假设存在一个最优调度 $\mathcal{S}^*$ 能最小化总工期，但它违反了 *Smallest-RLI-First* 规则。这意味着在某个时刻 $t \in I_{\text{stall}}$ ，调度器把带宽分配给了某条 RLI 更大的流 y ，而一条 RLI 更小的流 x 仍在等待。

具体而言，由于 t 属于停顿区间，必然存在某条待完成流 x 满足 $\text{RLI}(x) = 0$ （即其阻塞当前层）。而违反规则意味着在区间 $[t, t + \delta]$ 内， $\mathcal{S}^*$ 实际服务的是某条流 y ，且 $\text{RLI}(y) > \text{RLI}(x) = 0$ 。我们通过如下交换构造一个新调度 $\mathcal{S}'$ ：

Algorithm 1 请求间调度

```

1: procedure INTERSCHEDULING( $\mathcal{B}, M, B$ )
   ▶ input  $\mathcal{B}, M$ : Batch 集合与流量矩阵
   ▶ input  $B$ : 全局总丢弃预算
   ▶ output  $\sigma, \mathcal{H}$ 
2:    $S(\cdot) \leftarrow 0$  ▶ 来自高优先级 batch 的干扰
3:    $\mathcal{P} \leftarrow \emptyset$ ;  $\mathcal{H} \leftarrow \emptyset$ 
   ▶ 步骤 1: 按有效截止时间排序
4:    $\sigma \leftarrow \text{SORTBYRED}(\mathcal{B})$ 
5:   从  $M$  预计算负载向量  $L$ 
6:   for  $k \leftarrow 1$  to  $|\sigma|$  do
7:     令  $i \leftarrow \sigma[k]$ 
8:      $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{B}_i$  ▶ 加入候选池
9:      $\widehat{F}_i \leftarrow \text{ESTFINISHTIME}(S, L_i)$ 
   ▶ 步骤 2: 可行性检查
10:    while  $\widehat{F}_i > D_i^{Lo}$  and  $|\mathcal{H}| < B$  and  $\mathcal{P} \neq \emptyset$  do
11:       $u^* \leftarrow \arg \max_u (S(u) + L_i(u))$  ▶ 瓶颈
   ▶ 步骤 3: 选择性剪枝
12:       $r^* \leftarrow \arg \max_{r \in \mathcal{P}} \text{Load of } r \text{ on } u^*$ 
13:       $\mathcal{H} \leftarrow \mathcal{H} \cup \{r^*\}$ ;  $\mathcal{P} \leftarrow \mathcal{P} \setminus \{r^*\}$ 
14:      if  $r^* \in \mathcal{B}_i$  then ▶ 丢弃当前 batch 中的请求
15:         $L_i(\cdot) \leftarrow L_i(\cdot) - \ell_{r^*}(\cdot)$ 
16:      else ▶ 丢弃更高优先级 batch 中的请求
17:         $S(\cdot) \leftarrow S(\cdot) - \ell_{r^*}(\cdot)$ 
18:         $\widehat{F}_i \leftarrow \text{ESTFINISHTIME}(S, L_i)$ 
19:         $S(\cdot) \leftarrow S(\cdot) + L_i(\cdot)$  ▶ 更新
20:    return  $\sigma, \mathcal{H}$ 

```

- (1) **交换**：在 $\mathcal{S}'$ 中，将区间 $[t, t + \delta]$ 分配给流 x ，而不是流 y ；
- (2) **对停顿的影响**：由于 x 更早得到服务，当前层 L_{curr} 的依赖会提前 δ 个时间单位满足（假设 x 是瓶颈依赖）。这会严格缩短当前 I_{stall} 的持续时间，并提前下一段计算阶段（也因此提前下一段 I_{overlap} ）的开始时间；
- (3) **y 的可行性**：流 y 被从时刻 t 挪走。然而，由于 $\text{RLI}(y) > 0$ ，它直到未来某层才真正需要。 x 的完成会触发当前层计算 $C_{L_{\text{curr}}}$ 的开始，从而创造一个新的重叠区间。我们可以安全地把 y 的传输移入这个新产生的重叠区间。由于 y 的截止要求严格晚于 x ，这种延后不会违反任何依赖关系。

结论。构造得到的调度 $\mathcal{S}'$ 通过把一部分 I_{stall} 转换成 I_{overlap} ，从而减少了累计停顿时长。对所有违反情形重复这一交换论证即可证明：严格优先最小 RLI 的调度（立即服务 $\text{RLI} = 0$ 流以最小化停顿，并利用重叠区间服务 $\text{RLI} > 0$ 流）能够达到最小总工期。□

B 稳健请求间调度的完整算法

算法 1 展示了整体调度工作流。本节进一步阐释其调度逻辑，并给出优先级分配的完整数学形式以及时延估计与执行机制的更多实现细节。

该节介绍一种面向不精确松弛度与过载场景的稳健调度算法。整个 workflow 由 batch 到达与离开事件触发，并分为三个阶段。

步骤 1：通过稳健有效截止时间进行排序。 为了缓解“Piggyback Effect”，调度器采用第 4.4.2 节定义的稳健有效截止时间（RED）指标。优先级计算包含两个阶段。首先，对于包含 n 个请求的 batch $\mathcal{B}$ ，系统先按截止时间排序，得到 $d_1 \leq d_2 \leq \dots \leq d_n$ 。随后进行一次线性扫描，找到使相邻截止时间间隔最大的分割点 $k^* = \arg \max_{1 \leq k < n} (d_{k+1} - d_k)$ 。这一过程可动态地将离群请求（紧集）与主体请求（松集）区分开来。基于这一划分，调度器结合子 batch 截止时间与离群比例 f 计算 RED 分数。最后，所有活跃 batch 会被组织进一个按照 RED 升序排列的全局优先队列中。这样可以在保留真正紧急工作负载优先级的同时，有效过滤掉由少数离群点造成的短暂“虚高紧迫性”。

步骤 2：最坏情形可行性检查。 完成优先级排序后，我们执行准入控制检查，以避免“Black Hole Effect”。我们在“不同 batch 之间完全不重叠”的最坏假设下，通过严格累积延迟来估计完成时间 $\hat{T}_i$ 。计算时延视为确定性，可依据 Transformer 模型的静态计算图和目标硬件上的离线剖析得到 [5, 70, 106]。通信时延则通过分析 batch 的流量矩阵，识别网络中的瓶颈端口 p^* 来估计；延迟计算方式为该端口上的累计负载除以其带宽。对于如 Mixture-of-Experts (MoE) 这类 token 路由由运行时决定的动态架构，我们依赖历史路由统计。已有工作 [40] 表明，尽管单次流量矩阵噪声较大，但端到端时延预测误差通常仍控制在 20% 以内。对于我们的调度粒度而言，这样的精度已足够支撑面向目标截止时间 ($D_{\min}^{Lo}$) 的可行性约束。

步骤 3：选择性剪枝与软执行。 如果一个 batch 未通过可行性检查（即 $\hat{T}_i > D_{\min}^{Lo}$ ），系统就会触发外科手术式剪枝机制。我们识别出对瓶颈端口 p^* 贡献最大负载的具体请求，并迭代地将其从可行集合中移除，直到预测时延重新满足截止时间。关键的是，这种剪枝采用的是一种软执行策略。调度器不会立刻丢弃被剪枝请求，而是将它们降级到一个“双队列系统”中的“Scavenger”优先级类别。主队列（Main Queue）采用严格优先服务，而 Scavenger 队列则只在检测到网络空闲、或实际运行时延低于悲观估计时机会性地被服务。该混合策略既能在最坏预测条件下维持系统稳定，又能在拥塞并未真正发生时回收资源、尽可能提高吞吐。

总结。 需要强调的是，这一 workflow 内部具有清晰的设计层次。RED 指标是首要的决策依据，它控制全局发放顺序，在正常到中等负载下最大化服务级目标（SLO）满足率。可行性检查与选择性剪枝则仅作为互补性的保险机制，在病态过载场景下启动，以防止资源耗尽。这样的职责分离确保了调度器在绝大多数情况下保持稳定、可预测——主要依赖稳健排序来做决策——同时在极端争用不可避免时，仍具备平滑降级的能力。